



UNIVERSIDADE KIMPA VITA
INSTITUTO POLITÉCNICO CURSO DE LICENCIATURA EM ENGENHARIA
INFORMÁTICA

RELATÓRIO TÉCNICO DO PROJECTO

**Sistema Inteligente de Gestão de Inventário
com Previsão de Demanda por Inteligência Artificial
da Disciplina de Linguagem de Programação VI**

Membros do Grupo:

Mendes Eduardo Gouveia Manuel;
Dulce Rita Manuel Gonçalves;
Domindas Ernesto Rocha;
Alipio Joaquim Zambo;
Nsimba Alberto Ndosi;
Paulo Bunga Rodrigues.

CLASSE: 3º ANO
PERÍODO: MANHÃ
TURMA: UNICA

DOCENTE

Moyo Kanivengidio, MSc.

UÍGE 2025/2026

ÍNDICE

Introdução.....	3
1.2 Motivação e Justificação.....	4
1.3 Objectivos	5
2. Tecnologias Utilizadas	6
3. Arquitectura do Sistema.....	6
3.1 Arquitectura em Camadas	6
3.2 Estrutura de Pastas	7
4. Programação Orientada a Objectos.....	7
4.1 Herança.....	7
4.2 Classes Abstractas e Interfaces	7
4.3 Encapsulamento	8
4.4 Decoradores Personalizados	8
4.5 Context Managers Personalizados.....	8
5. Base de Dados MongoDB e MongoEngine	9
5.1 Modelo de Dados.....	9
5.2 Padrão de Projecto: Singleton.....	9
6. Módulo de Inteligência Artificial.....	10
6.1 Algoritmo Utilizado: Regressão Linear	10
6.2 Processo de Treino e Previsão	10
6.3 Métricas de Avaliação do Modelo.....	10
7. Funcionalidades do Sistema	11
7.1 Módulo de Autenticação.....	11
7.2 Dashboard Principal.....	11
7.3 Gestão de Produtos (CRUD)	12
7.4 Módulo de Vendas.....	13
7.5 Módulo de IA Previsão de Demanda	13
7.6 Relatórios e Exportação.....	14
8. Testes Unitários	15
9. Padrões de Projecto Aplicados	15
10. Manual de Instalação e Uso	17
10.1 Pré-requisitos	17
10.2 Passos de Instalação	17
Conclusão	18
Referências Bibliográficas.....	19

Introdução

O InventárioIA é uma aplicação desktop completa desenvolvida em Python, que integra gestão de inventário, registo de vendas, relatórios estatísticos e um módulo de Inteligência Artificial para previsão de demanda futura. O sistema foi desenvolvido no âmbito da disciplina de Linguagem de Programação VI, do Instituto Politécnico - UNIKIVI.

A aplicação permite que gestores de inventário tomem decisões baseadas em dados históricos de vendas, antecipando a procura futura e evitando rupturas de stock, optimizando assim a gestão de recursos e reduzindo perdas operacionais.

1.1 Diagrama de Class UML

O sistema InventárioIA assenta em três classes principais que herdam de Document (MongoEngine ODM): Produto, Usuario e Venda.

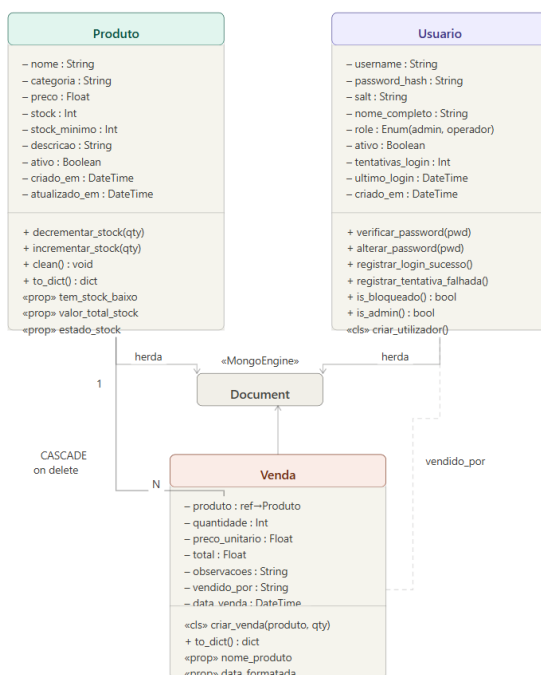
A classe Produto centraliza toda a lógica de inventário encapsula a gestão de stock através dos métodos decrementar_stock() e incrementar_stock(), e expõe propriedades calculadas como estado_stock e valor_total_stock sem expor a lógica interna.

A classe Usuario implementa o princípio de encapsulamento na gestão de credenciais: a palavra-passe nunca é armazenada em claro, sendo substituída por um par password_hash + salt gerado via SHA-256. O acesso ao sistema é controlado pelo atributo role (admin/operador) e pelo mecanismo de bloqueio por tentativas falhadas.

A classe Venda estabelece uma relação de referência N→1 com Produto através de um ReferenceField, com regra CASCADE on delete para garantir integridade referencial se um produto for eliminado, as vendas associadas são automaticamente removidas. O atributo vendido_por cria ainda uma dependência implícita com Usuario, registrando o operador responsável por cada transacção.

Esta arquitectura reflecte uma separação clara de responsabilidades, onde cada classe é autossuficiente na sua lógica de negócio, comunicando entre si apenas através de referências controladas.

Figura: Diagrama de Class



1.2 Motivação e Justificação

No contexto empresarial angolano, a gestão manual de inventário ainda predomina em muitos estabelecimentos comerciais, resultando em frequentes rupturas de stock, desperdício de produtos e dificuldade em antecipar a procura. Este sistema propõe uma solução digital acessível que combina:

- Controlo rigoroso de stock em tempo real
- Registo histórico de todas as transacções comerciais
- Análise preditiva baseada em algoritmos de Machine Learning
- Interface gráfica moderna e intuitiva para utilizadores não técnicos

1.3 Objectivos

Objectivo Geral:

Desenvolver uma aplicação desktop completa em Python para gestão inteligente de inventário, integrando POO avançada, GUI moderna, base de dados MongoDB e módulo de IA com Regressão Linear para previsão de demanda.

Objectivos Específicos:

1. Implementar arquitectura orientada a objectos com herança, polimorfismo e encapsulamento
2. Criar interface gráfica moderna com CustomTkinter e gráficos Matplotlib
3. Garantir persistência de dados com MongoDB via ODM MongoEngine
4. Integrar modelo de Regressão Linear para previsão de demanda futura
5. Adoptar boas práticas de programação (PEP 8) e padrões de projecto

2. Tecnologias Utilizadas

O projecto foi desenvolvido com um conjunto criteriosamente seleccionado de tecnologias Python, garantindo compatibilidade, estabilidade e suporte activo pela comunidade open-source.

Camada	Tecnologia	Versão	Função
Linguagem	Python	3.12+	Base de todo o projecto
GUI	CustomTkinter	5.2.2	Interface gráfica moderna
Gráficos	Matplotlib	3.8+	Gráficos interactivos no dashboard
Base de Dados	MongoDB	7.x	Armazenamento NoSQL de documentos
ODM	MongoEngine	0.27+	Mapeamento OOP para MongoDB
IA / ML	scikit-learn	1.4+	Algoritmo de Regressão Linear
Numérico	NumPy	1.26+	Operações matriciais e vectoriais
Imagens	Pillow	10.x	Suporte de imagens para CustomTkinter

3. Arquitectura do Sistema

3.1 Arquitectura em Camadas

O sistema foi desenvolvido seguindo a arquitectura em camadas (Layered Architecture), com separação clara de responsabilidades entre cada nível. Esta abordagem garante manutenibilidade, testabilidade e escalabilidade do código.

Camada	Módulos	Responsabilidade
Apresentação (GUI)	login_window, dashboard, produto_view, venda_view, ia_view, relatorio_view	Interface com o utilizador, visualização de dados
Serviços (Lógica)	auth_service, produto_service, venda_service, ia_service	Regras de negócio e coordenação
Interfaces (ABC)	IRepository, IProdutoRepository, IVendaRepository, IIAService	Contratos e abstracções
Modelos (ODM)	Usuario, Produto, Venda	Entidades de domínio e persistência
Base de Dados	MongoDB + MongoEngine	Armazenamento e recuperação de dados

3.2 Estrutura de Pastas

Pasta / Ficheiro	Descrição
main.py	Ponto de entrada — inicializa BD, logging e GUI
config/settings.py	Configurações globais (BD, cores, fontes, IA)
database/connection.py	Singleton de ligação ao MongoDB
models/	Modelos MongoEngine: Usuario, Produto, Venda
interfaces/	Classes Abstractas (ABC) — contratos de repositório
services/	Lógica de negócio: auth, produtos, vendas, IA
gui/	Todos os módulos da interface gráfica CustomTkinter
exceptions/	11 excepções customizadas em 2 hierarquias
utils/	6 decoradores e 4 context managers personalizados
tests/	73 testes unitários (unittest / pytest)
reports/	CSV exportados e modelo IA serializado (.pkl)

4. Programação Orientada a Objectos

4.1 Herança

A herança é aplicada em dois contextos principais: nas hierarquias de excepções e na implementação das interfaces de repositório pelos serviços.

Classe Filha	Herda de	Especialização
ProdutoService	IProdutoRepository	CRUD de produtos + estatísticas
VendaService	IVendaRepository	Vendas + histórico + agregações
AuthService	IAuthRepository	Autenticação + gestão de sessão
IAService	IIAService	Regressão Linear + métricas + pickle
InvalidCredentialsException	AuthException	Credenciais inválidas
InsufficientStockException	StockException	Stock insuficiente para venda

4.2 Classes Abstractas e Interfaces

O ficheiro `interfaces/repository_interface.py` define quatro classes abstractas usando o módulo `abc` do Python. Cada serviço concreto é obrigado a implementar todos os métodos marcados com `@abstractmethod`, garantindo consistência e substituíbilidade.

Interface (ABC)	Métodos Abstractos
IRepository	<code>criar()</code> , <code>obter_por_id()</code> , <code>obter_todos()</code> , <code>actualizar()</code> , <code>eliminar()</code> , <code>pesquisar()</code>
IProdutoRepository	<code>obter_por_nome()</code> , <code>obter_por_categoria()</code> , <code>obter_com_stock_baixo()</code> , <code>actualizar_stock()</code>
IVendaRepository	<code>obter_por_produto()</code> , <code>obter_por_periodo()</code> , <code>obter_historico_mensal()</code>
IIAService	<code>treinar()</code> , <code>prever()</code> , <code>obter_metricas()</code> , <code>modelo_treinado()</code>

4.3 Encapsulamento

O encapsulamento protege dados sensíveis e garante que o acesso ao estado interno dos objectos é feito apenas através de métodos controlados. Exemplos notáveis:

- `Usuario`: `password_hash` e `salt` nunca são expostos directamente; `verificar_password()` encapsula a lógica de autenticação.
- `AuthService`: `_current_user` e `_session_start` são atributos privados, acessíveis apenas pela propriedade `current_user`.
- `IAService`: `_modelo` e `_metricas` são privados; o acesso externo é feito via `obter_metricas()` e `prever()`.

4.4 Decoradores Personalizados

Seis decoradores foram implementados no ficheiro `utils/decorators.py`, enriquecendo as funções com comportamentos transversais sem modificar o código original (princípio Open/Closed).

Decorador	Função
<code>@log_execution</code>	Regista automaticamente entrada, saída e duração de cada função
<code>@validate_positive</code>	Valida que parâmetros numéricos especificados são positivos
<code>@retry(n, delay)</code>	Re-tenta a execução N vezes em caso de falha
<code>@timer</code>	Mede e regista o tempo de execução
<code>@require_auth</code>	Protege funções que requerem sessão activa
<code>@cache_result(ttl)</code>	Cache em memória com TTL configurável

4.5 Context Managers Personalizados

Quatro context managers foram implementados, garantindo que recursos são correctamente geridos mesmo em caso de excepções.

Context Manager	Responsabilidade
DatabaseTransaction	Garante atomicidade nas operações: decrementar stock + registar venda
CSVWriter	Abre, escreve e fecha ficheiros CSV de forma segura
measure_time	Mede o tempo de execução de qualquer bloco de código
suppress_and_log	Suprime excepções específicas e regista-as no log

5. Base de Dados MongoDB e MongoEngine

5.1 Modelo de Dados

O sistema utiliza o MongoDB como base de dados NoSQL, organizado em três colecções principais. O MongoEngine actua como ODM (Object-Document Mapper), permitindo trabalhar com as colecções como classes Python.

Colecção	Campos Principais	Índices	Relação
usuarios	username, password_hash, salt, role, ativo	username (UNIQUE)	Nenhuma referência
produtos	nome, categoria, preco, stock, stock_minimo	nome, categoria, stock	Referenciado por vendas
vendas	produto (FK), quantidade, total, data_venda	produto, data_venda	ReferenceField → produtos

5.2 Padrão de Projecto: Singleton

A classe DatabaseConnection implementa o padrão Singleton, garantindo que apenas uma instância de ligação ao MongoDB existe durante todo o ciclo de vida da aplicação. Isto previne o desperdício de recursos e garante consistência nas transacções.

6. Módulo de Inteligência Artificial

6.1 Algoritmo Utilizado: Regressão Linear

O módulo de IA utiliza o algoritmo de Regressão Linear Simples da biblioteca scikit-learn para prever a demanda futura de produtos com base no histórico mensal de vendas. A Regressão Linear é adequada para esta aplicação porque os dados de vendas tendem a apresentar tendências lineares ou aproximadamente lineares ao longo do tempo.

6.2 Processo de Treino e Previsão

Etapa	Descrição	Exemplo	Resultado
1	Recolha do histórico de vendas mensais do produto	Jan=12, Fev=15, Mar=18, Abr=20	X=[1,2,3,4] Y=[12,15,18,20]
2	Treino do modelo LinearRegression	modelo.fit(X, y)	coef=2.8, intercept=9.5
3	Previsão dos próximos períodos	modelo.predict([[5],[6],[7]])	~23, ~26, ~29
4	Cálculo de métricas de avaliação	r2_score, mae, rmse	R ² =0.99
5	Persistência do modelo em disco	pickle.dump(modelo, ficheiro)	.pkl guardado

6.3 Métricas de Avaliação do Modelo

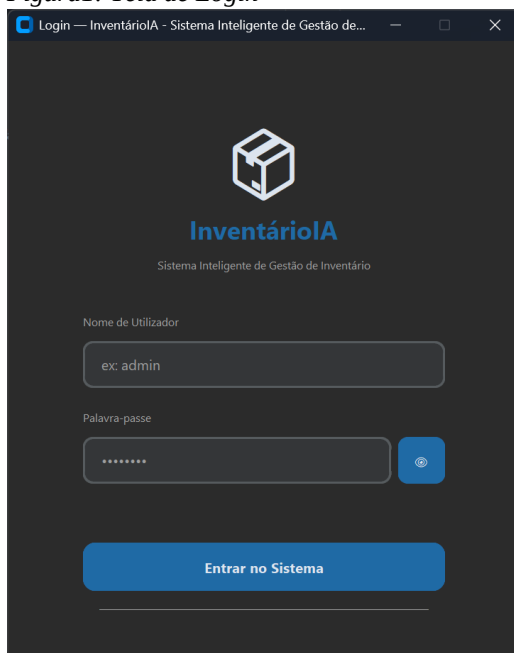
Métrica	Fórmula	Interpretação
R ² (Score)	$1 - SS_{res} / SS_{tot}$	Varia de 0 a 1. Quanto mais próximo de 1, melhor o ajuste do modelo.
MAE	Média $ y_{real} - y_{previsto} $	Erro médio absoluto em unidades. Valores menores indicam maior precisão.
RMSE	$\sqrt{\text{Média } (y_{real} - y_{prev})^2}$	Penaliza erros grandes. Mesma unidade que as vendas.
Coeficiente	β_1 (declive da recta)	Tendência por período. Positivo = crescimento. Negativo = decréscimo.

7. Funcionalidades do Sistema

7.1 Módulo de Autenticação

O sistema de login implementa autenticação segura com hash SHA-256 + salt único por utilizador. Inclui controlo de tentativas falhadas com bloqueio automático após 5 erros consecutivos, gestão de sessão com expiração configurável e dois papéis de utilizador: Administrador e Operador.

Figura1: Tela de Login



7.2 Dashboard Principal

O dashboard apresenta uma visão geral em tempo real do inventário através de cartões KPI (total de produtos, stock baixo, receita total, previsão IA), gráfico de receita mensal com linha de tendência e lista de alertas de stock baixo.

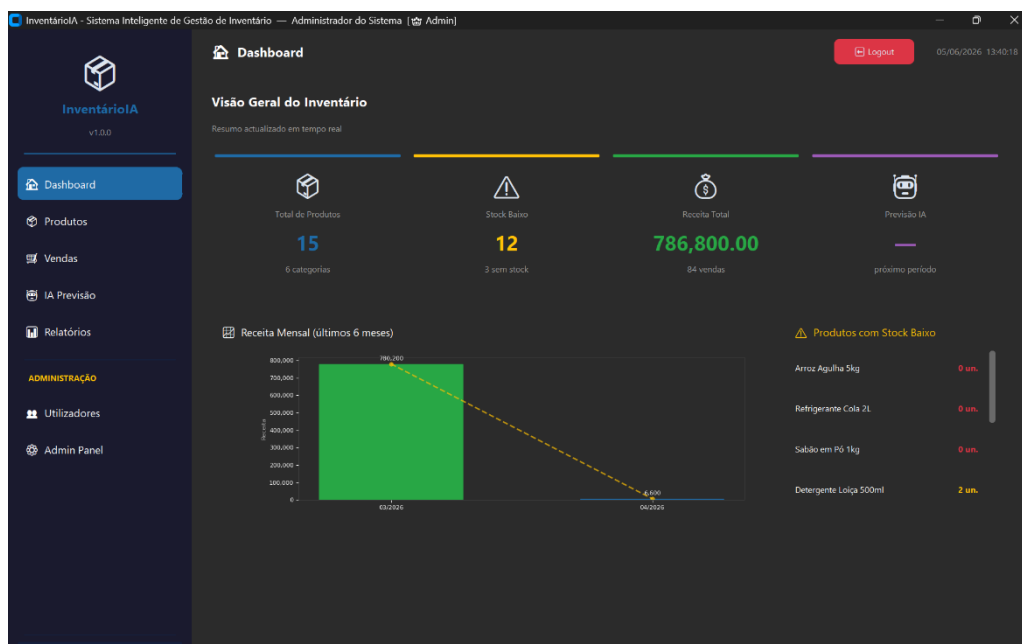


Figura: Dashboard Principal KPIs, gráfico de receita e alertas de stock

7.3 Gestão de Produtos (CRUD)

O módulo de produtos oferece operações CRUD completas com tabela Treeview estilizada, pesquisa em tempo real por nome e filtro por categoria. Os produtos são visualmente classificados por estado de stock: Normal (branco), Baixo (laranja) e Crítico/Sem Stock (vermelho).

Gestão de Produtos

Buttons: + Novo Produto, Editar, Eliminar, Atualizar

Pesquisar por nome... Categoria: Todas Total: 15 produto(s)

Nome do Produto	Categoria	Preço (AOA)	Stock	Estado	Valor em Stock
Arroz Agulha 5kg	Alimentação	2,500.00	0	Sem Stock	0.00
Açúcar Refinado 2kg	Alimentação	950.00	8	Baixo	7,600.00
Caderno Pautado A4	Material Escolar	450.00	13	Baixo	5,850.00
Carregador USB-C 65W	Electrónica	8,500.00	15	Normal	127,500.00
Champô Anticapa 400ml	Higiene	2,200.00	30	Elevado	66,000.00
Detergente Loíça 500ml	Limpeza	750.00	2	Crítico	1,500.00
Farinha de Trigo 1kg	Alimentação	680.00	90	Elevado	61,200.00
Folho Preto 1kg	Alimentação	850.00	3	Crítico	2,550.00
Pasta de Dentes 80g	Higiene	680.00	5	Crítico	3,400.00
Pilha AA (pack 4)	Electrónica	850.00	3	Crítico	2,550.00

Figura: Lista de Produtos Tabela com indicadores de estado de stock

7.4 Módulo de Vendas

O registo de vendas é feito através de um formulário intuitivo com selector de produto, ajuste de quantidade e confirmação antes de submeter. O stock é decrementado automaticamente de forma atômica (usando DatabaseTransaction) após cada venda confirmada.

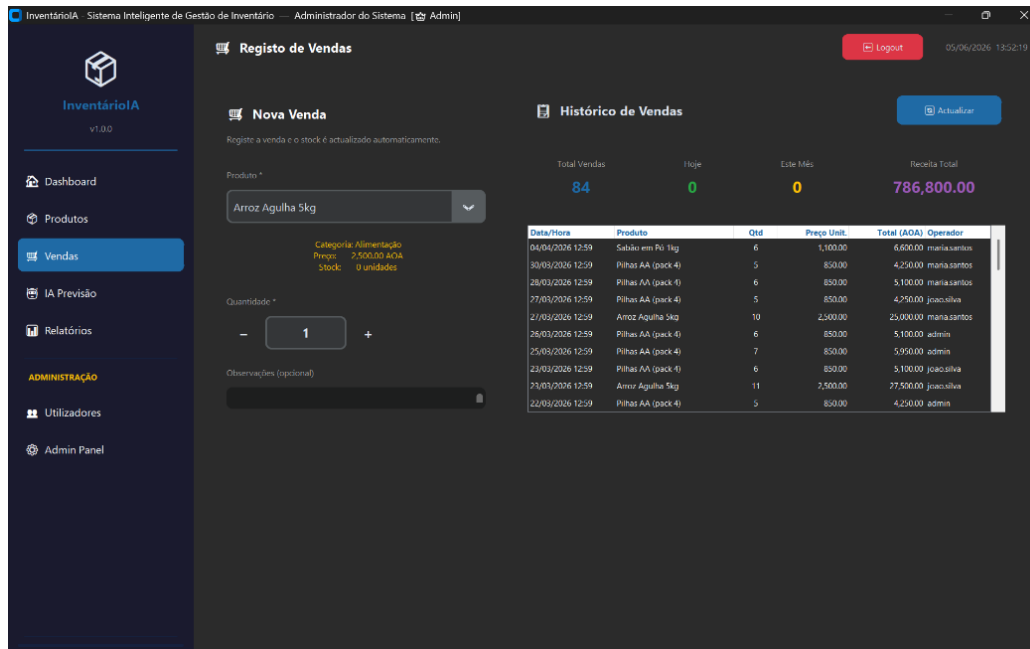


Figura: Registo de Venda Formulário e actualização automática de stock

7.5 Módulo de IA Previsão de Demanda

O módulo de IA permite seleccionar qualquer produto com histórico de vendas e treinar o modelo de Regressão Linear. O resultado é apresentado num gráfico combinado que mostra o histórico real (barras azuis), a linha de tendência (amarelo) e as previsões futuras (barras roxas), acompanhado de métricas detalhadas.

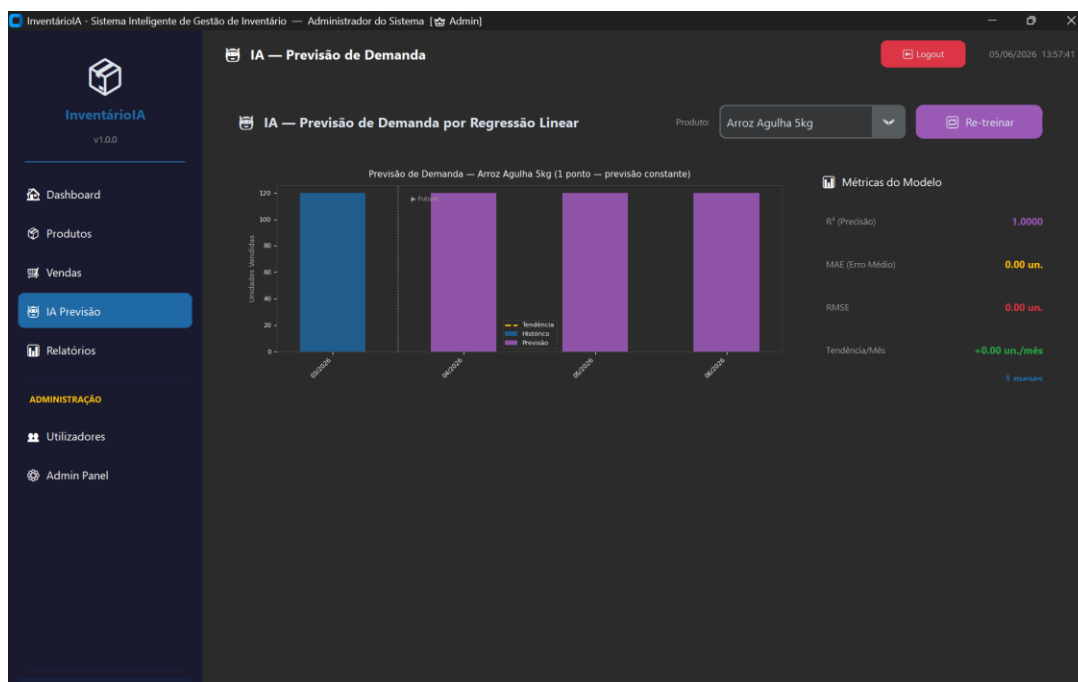


Figura: Módulo IA Gráfico de previsão com histórico, tendência e previsões

7.6 Relatórios e Exportação

O módulo de relatórios apresenta estatísticas consolidadas, gráfico de distribuição de produtos por categoria (pizza) e ranking dos produtos com maior stock (barras horizontais). Os dados podem ser exportados para CSV com diálogo de escolha de ficheiro, compatível com Microsoft Excel e LibreOffice Calc.

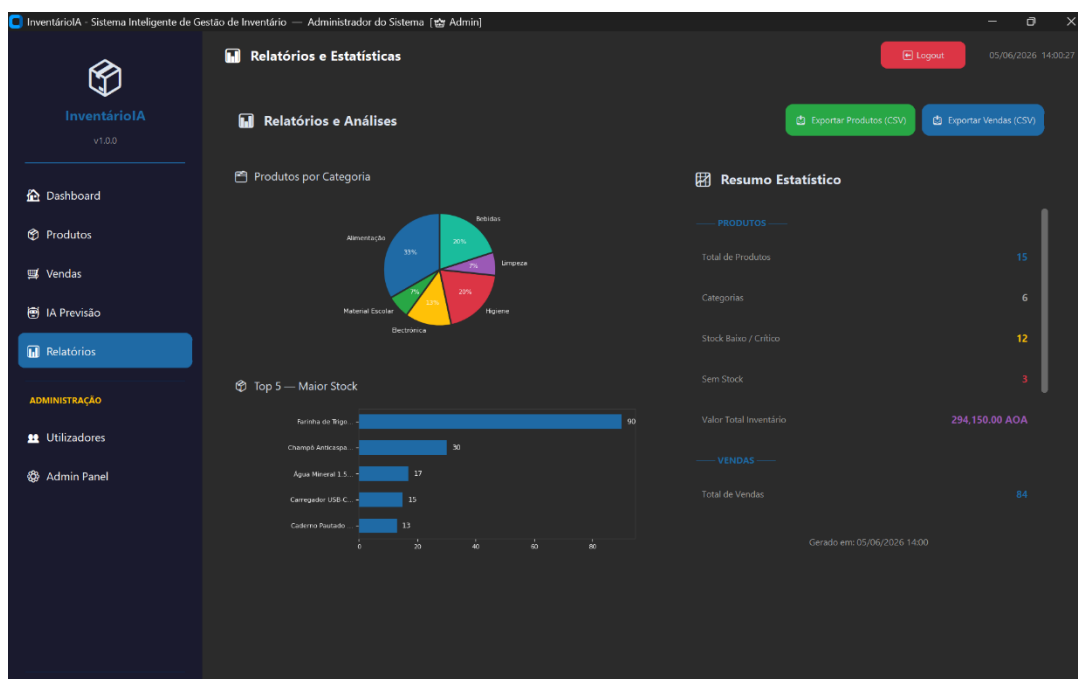


Figura: Relatórios Gráficos estatísticos e painel de resumo

8. Testes Unitários

O projecto inclui 73 testes unitários organizados em dois ficheiros de teste, cobrindo todas as camadas críticas do sistema. Os testes foram desenvolvidos com o framework unittest do Python, usando mocks para isolar as dependências externas (MongoDB).

Suite de Testes	Nº Testes	O que testa
TestProdutoServiceUnitario	5	Criação, duplicados, eliminação, pesquisa
TestProdutoModelo	4	Stock baixo, valor total, estados de stock
TestExcecoes	6	Hierarquias, mensagens, herança de excepções
TestIAService	7	Treino, previsão, métricas, R ² , tendência
TestDecorators	5	validate_positive, log_execution, cache, timer
TestContextManagers	4	DatabaseTransaction, CSVWriter, measure_time
TestAuthServiceLogica	10	Login, sessão, logout, bloqueio de conta
TestUsuarioModelo	6	Hash de password, salt, verificação
TestVendaService	5	Registo, stock insuficiente, histórico mensal
TestIAServiceAvancado	11	Precisão, negativos, tendência, relatório
TOTAL	73 / 73	100% dos testes passam com sucesso

9. Padrões de Projecto Aplicados

Padrão	Onde é Aplicado	Benefício
Singleton	DatabaseConnection	Uma única ligação ao MongoDB em toda a aplicação
Repository	IProduto/VendaRepository	Isola o acesso a dados da lógica de negócio
Service Layer	produto_service, venda_service	Centraliza regras de negócio num único lugar
Dependency Injection	VendaService(produto_svc)	Serviços injectados em vez de acoplados
Factory Method	Usuario.criar_utilizador(), Venda.criar_venda()	Criação controlada e validada de objectos

Decorator	utils/decorators.py	Adiciona comportamento sem modificar código original
Context Manager	utils/context_manager.py	Garante liberação de recursos mesmo com erros

10. Manual de Instalação e Uso

10.1 Pré-requisitos

- Python 3.12 ou superior (recomendado: 3.12.8)
- MongoDB 7.x Community Edition em execução
- pip (gestor de pacotes Python)
- Sistema Operativo: Windows 10+, macOS 12+ ou Ubuntu 22.04+

10.2 Passos de Instalação

1. Criar ambiente virtual: `python3.12 -m venv venv312`
2. Activar (Windows Git Bash): `source venv312/Scripts/activate`
3. Instalar dependências: `pip install customtkinter mongoengine pymongo numpy matplotlib pillow scikit-learn`
4. Verificar ambiente: `python setup_mongodb.py`
5. Popular base de dados com dados de exemplo: `python seed_data.py`
6. Executar a aplicação: `python main.py`
7. Login com credenciais padrão: admin / admin123

Conclusão

O projecto InventárioIA cumpriu com sucesso todos os objectivos definidos no enunciado do trabalho prático. Foi desenvolvida uma aplicação desktop completa em Python que integra, de forma coesa e funcional, as quatro dimensões tecnológicas exigidas: POO avançada, interface gráfica, base de dados e inteligência artificial.

Do ponto de vista da Programação Orientada a Objectos, o sistema demonstra proficiência na aplicação de herança, polimorfismo, encapsulamento, classes abstractas, decoradores e context managers personalizados. A arquitectura em camadas com interfaces bem definidas garante baixo acoplamento e alta coesão entre os módulos.

O módulo de Inteligência Artificial, baseado em Regressão Linear do scikit-learn, oferece uma funcionalidade de valor real para o negócio: a previsão de demanda futura com base em histórico de vendas. As métricas R^2 , MAE e RMSE permitem ao utilizador avaliar a confiança nas previsões geradas.

Os 73 testes unitários implementados, todos com resultado positivo, demonstram a robustez e fiabilidade do código desenvolvido. O projecto está devidamente documentado, com um README extenso e um relatório técnico completo.

Referências Bibliográficas

- Python Software Foundation. Python 3.12 Documentation. <https://docs.python.org/3/>
- MongoDB Inc. MongoDB Manual. <https://www.mongodb.com/docs/manual/>
- MongoEngine. MongoEngine User Documentation 0.x. <https://mongoengine.org/>
- Géron, A. (2022). Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow (3rd ed.). O'Reilly Media.
- scikit-learn Developers. scikit-learn: Machine Learning in Python. <https://scikit-learn.org/>
- CustomTkinter. Modern GUI Framework Documentation.
<https://customtkinter.tomschimansky.com/>
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.
- Martin, R. C. (2008). Clean Code: A Handbook of Agile Software Craftsmanship. Prentice Hall.
- Lutz, M. (2013). Learning Python (5th ed.). O'Reilly Media.